

WEEK 9

Task 1

Skill objective:

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

void execute_input_redirect(const char *cmd, const char *infile) {
    printf("[Task 1] Parsing Input Redirection: '%s < %s'\n", cmd, infile);
    pid_t pid = fork();
    if (pid == 0) {
        int fd = open(infile,
            O_RDONLY);
        if (fd < 0) {
            perror("Error opening input
            file");
            exit(EXIT_FAILURE);
        }
        dup2(fd,
            STDIN_FILENO);
        close(fd);
        execlp(cmd, cmd, NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }
    int status;
    waitpid(pid, &status, 0);
}

int main() {
    /* Setup input test file */
    FILE *f =
        fopen("test_input.txt", "w");
    fprintf(f, "Line 1: Shell input
    redirection stream\n"
        "Line 2: Systems programming
    continuous assessment\n"
        "Line 3: Verified end-of-file
    transmission\n");
    fclose(f);

    execute_input_redirect("cat", "test_input.txt");
    printf("[Task 1] Testing missing file error handling:\n");
    execute_input_redirect("cat", "nonexistent_file.txt");
    return 0;
}
```

Output:

```
charanteja_a@charan:~$ nano exp9s.c
charanteja_a@charan:~$ gcc exp9s.c -o exp9s
charanteja_a@charan:~$ ./exp9s
[Task 1] Parsing Input Redirection: 'cat < test_input.txt'
Line 1: Shell input redirection stream
Line 2: Systems programming continuous assessment
Line 3: Verified end-of-file transmission
[Task 1] Testing missing file error handling:
[Task 1] Parsing Input Redirection: 'cat < nonexistent_file.txt'
Error opening input file: No such file or directory
charanteja_a@charan:~$
```

Task 2

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

Skill objective:**Implementation / work:**

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

void execute_output_redirect(const char *outfile) {    printf("[Task 2]
Executing Output Redirection: 'ls -l > %s'\n", outfile);    pid_t pid =
fork();    if (pid == 0) {        int fd = open(outfile, O_WRONLY | O_CREAT
| O_TRUNC, 0644);        if (fd < 0) { perror("open outfile"); exit(1); }
dup2(fd, STDOUT_FILENO);        close(fd);        execlp("ls", "ls", "-l",
NULL);        exit(1);    }    waitpid(pid, NULL, 0); }

int main() {    const char *target =
"output_target.txt";
execute_output_redirect(target);

    printf("[Task 2] Verifying redirected file content and permissions:\n");
system("ls -l output_target.txt");    printf("Content excerpt from
output_target.txt:\n");    system("head -n 3 output_target.txt");    return
0; } Output:
```

```
charanteja_a@charan:~$ nano exp9ss.c
charanteja_a@charan:~$ gcc exp9ss.c -o exp9ss
charanteja_a@charan:~$ ./exp9ss
[Task 2] Executing Output Redirection: 'ls -l > output_target.txt'
[Task 2] Verifying redirected file content and permissions:
-rw-r--r-- 1 charanteja_a charanteja_a 8681 Oct  5 19:15 output_target.txt
Content excerpt from output_target.txt:
total 1264
-rw-r--r--  1 charanteja_a charanteja_a   798 Oct  5 18:40 9.1.c
-rw-----  1 charanteja_a charanteja_a     4 Oct  5 10:09 9.1.c.save
charanteja_a@charan:~$ |
```

Task 3

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

void append_to_file(const char *msg, const char *filename) {    pid_t
pid = fork();    if (pid == 0) {        int fd = open(filename,
O_WRONLY | O_CREAT | O_APPEND, 0644);        if (fd < 0) {
```

Skill objective:

```

perror("open append"); exit(1); }          dup2(fd, STDOUT_FILENO);
close(fd);          execlp("echo", "echo", msg, NULL);          exit(1);
}          waitpid(pid, NULL, 0); }

int main() {          const char *logfile =
"append_log.txt";          unlink(logfile); /*
clean start */

    printf("[Task 3] Writing base record...\n");
    append_to_file("Log Entry 1: Base system initialized", logfile);

    printf("[Task 3] Appending record via '>>' operator without truncating...\n");
    append_to_file("Log Entry 2: Network module started", logfile);          append_to_file("Log
Entry 3: Security audit passed", logfile);

    printf("Verified Cumulative File Content:\n");
    system("cat append_log.txt");          return 0; }

```

Output:

```

charanteja_a@charan:~$ nano exp9s3.c
charanteja_a@charan:~$ gcc exp9s3.c -o exp9s3
charanteja_a@charan:~$ ./exp9s3
[Task 3] Writing base record...
[Task 3] Appending record via '>>' operator without truncating...
Verified Cumulative File Content:
Log Entry 1: Base system initialized
Log Entry 2: Network module started
Log Entry 3: Security audit passed
charanteja_a@charan:~$ |

```

Task 4

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Implementation / work:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {          const char *err_file = "stderr_capture.log";          printf("[Task 4]
Redirecting stderr (fd 2) to '%s' using '2>'...\n", err_file);

    pid_t pid = fork();          if (pid == 0) {          int fd =
open(err_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);          if (fd <
0) { perror("open"); exit(1); }          dup2(fd, STDERR_FILENO);
close(fd);          /* Execute command generating deliberate stderr

```

Skill objective:

```
output */          execlp("ls", "ls", "missing_dir_xyz", NULL);
exit(1);           }      waitpid(pid, NULL, 0);

    printf("[Task 4] Error stream isolated. Verifying '%s':\n", err_file);
system("cat stderr_capture.log");    return 0; } Output:
```

```
charanteja_a@charan:~$ nano exp9s4.c
charanteja_a@charan:~$ gcc exp9s4.c -o exp9s4
charanteja_a@charan:~$ ./exp9s4
[Task 4] Redirecting stderr (fd 2) to 'stderr_capture.log' using '2>'...
[Task 4] Error stream isolated. Verifying 'stderr_capture.log':
ls: cannot access 'missing_dir_xyz': No such file or directory
charanteja_a@charan:~$ |
```